CHRIST
(DEEMED TO BE UNIVERSITY)
DELHI-NCR, INDIA

NLDL Programming

MCA-575

Assignment – 09

BY

HIMANSHU HEDA (24225013)

SUBMITTED TO

Dr. Manjula Shannhog

SCHOOL OF SCIENCES

2025-26

Importing the Libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score
from scipy.spatial.distance import cdist
import math
```

This cell generates a synthetic classification dataset.
- `make_classification` creates 300 samples with 2 informative features and 2 classes.
- The data is stored in `x` (features) and `y` (labels).

```
x , y = make_classification(
    n_samples=300,
    n_features=2,
    n_redundant=0,
    n_informative=2,
    n_clusters_per_class=2,
    n_classes=2,
    random_state=42
)
```

This cell displays the feature matrix `x`.
- Shows the generated feature values for all samples.

```
x
```

```
array([[-0.40517736, -0.51884623],
       [-0.48937373, -2.02835227],
       [ 1.20549953,  0.81408964],
       [ 0.86270883, -0.68933991],
       [ 1.55633721,  0.06675339],
       [-1.5010232 , -0.78301273],
       [-0.51056971,  0.62591976],
       [ 1.28471834,  2.10340158],
       [ 0.76733377, -1.10299711],
       [ 1.72305322,  1.15983583],
       [-1.74788575, -0.41869537],
       [ 0.98203471, -0.80359207],
       [-0.57233329,  0.6565228 ],
       [ 0.8053611 , -1.28464989],
       [-2.19841677,  1.94238253],
       [ 1.05061229,  1.10135077],
       [-0.69936114,  0.75945437],
```

[0.77597161, -1.23387951],
[0.93325921, -1.22891349],
[1.17170001, -1.06885555],
[-0.02177609, 2.025228],
[-1.66992286, -0.13014736],
[1.21164088, -0.72102924],
[0.95493192, -0.83121463],
[0.72571666, 1.72586366],
[-1.80230801, -1.12982433],
[0.4250635 , 1.01667061],
[-1.71999525, 1.56423673],
[2.66130799, 0.26055845],
[0.94913573, -0.5292874],
[-0.99683409, -0.72389266],
[-0.74457232, 0.1409602],
[0.16103282, 0.08771276],
[-2.18363538, 1.9153729],
[1.36062694, 2.46994871],
[-0.90653326, -1.17222446],
[0.42612593, 1.4395251],
[1.39513641, -0.57649564],
[1.17575757, -0.94007012],
[-1.09783325, -1.27552701],
[-0.79467835, -1.04546491],
[0.90010045, 1.5724083],
[-0.32375649, 0.47508067],
[0.80547073, -1.35091873],
[0.93916483, -1.33000495],
[-2.08331986, 1.87002253],
[-2.66793058, 2.31673559],
[1.34371609, 0.44765735],
[-0.66377587, -0.79666661],
[-1.37881744, -1.16748167],
[-0.57735993, -0.75006708],
[0.60408808, -0.26728809],
[0.9220511 , -1.23559176],
[-1.09285852, 1.09528274],
[-1.43080657, 1.33866147],
[-0.35665972, 0.49359616],
[-0.41224717, -1.89300279],
[0.88812951, -1.199256],
[1.68005259, 0.42815249],
[-0.00477984, 0.23589576],
[-0.11970876, 0.16748684],
[1.24549729, -0.53171271],
[1.23118719, 1.26219615],

[1.01961004, -1.34898418],
[0.23507404, 1.79896623],
[0.91448764, -1.07481215],
[2.42858463, 1.04765753],
[0.6983373 , -1.34225027],
[0.44250717, 1.62410081],
[-0.15402246, 0.71596991],
[-1.84390688, 1.656119],
[-1.90384565, 1.68740486],
[-0.99709252, 1.85377722],
[-0.73417001, 1.38586178],
[0.6143141 , 0.52011731],
[0.96768857, -1.25046094],
[-0.48703311, 0.61005116],
[0.92067433, 1.59124683],
[-0.50029417, -1.35101318],
[1.03917155, -1.74876956],
[-1.54457193, 1.42190908],
[-0.52989268, -0.54793059],
[-1.04034022, 1.03287851],
[0.38891062, 1.58998993],
[1.06291321, -0.78028019],
[1.78469526, 0.79641707],
[-1.92546594, -0.31340941],
[-0.95763068, -0.73485233],
[-1.03878184, 1.03749495],
[0.33534818, -0.01953164],
[-1.16850458, 1.13625788],
[-0.67977323, 0.74997242],
[1.23327762, 1.01427836],
[0.60899932, -1.94472136],
[1.20498315, -0.65886719],
[1.22785165, 1.39053846],
[-0.97901294, -1.98266619],
[1.07638095, 1.60722428],
[-0.9063357 , -0.28535436],
[1.21844334, 0.47151731],
[-1.60016165, -0.71283973],
[-1.02136518, -1.47972473],
[2.00418941, 1.1916392],
[-0.74871628, -0.51937891],
[1.56627376, 0.2718301],
[0.62972691, 1.64736862],
[1.51752315, 0.22966493],
[-0.81823937, -1.32981572],
[1.23097593, 1.25989926],

[-1.43907655, -0.71760113],
[-0.27786578, 0.05139204],
[-1.46036466, -1.97549711],
[-1.20973942, 1.15057337],
[1.32664861, -0.15728065],
[-1.36236534, 1.29123378],
[0.85750993, -1.13958989],
[-2.67122352, 2.30169281],
[-0.12544255, 0.31939392],
[-0.62337136, -1.32446303],
[0.35695937, 2.61681622],
[-2.38048008, 2.07545727],
[0.41095178, 0.90481935],
[0.73883661, -1.49904571],
[-0.40198408, 0.98101051],
[0.86140673, 1.16173323],
[-1.12822199, -0.36426636],
[-1.00503253, 1.00449638],
[-1.64995899, 1.50850236],
[0.48487861, -1.62269703],
[0.7988656 , -1.21237363],
[-1.82945505, -0.92734102],
[0.87079013, -1.3907574],
[-0.68415693, -1.15193494],
[1.12487537, -0.78165359],
[0.76941252, -1.87810278],
[-1.34933075, -0.80312239],
[-0.81943564, 0.85939709],
[1.30559428, -0.16366887],
[-0.0813886 , -1.51741197],
[-0.2109224 , 0.39454605],
[-0.96464992, -1.34495491],
[-1.3390232 , 1.27938694],
[-0.80379592, 0.83878785],
[1.774038 , 0.82476281],
[1.58084082, 0.34313214],
[1.59515083, 1.53042463],
[1.20224117, -0.62483658],
[-1.56612464, 1.4344788],
[-1.49116837, 1.39048489],
[1.0258755 , -0.51621948],
[1.80070544, 1.32349094],
[-1.40556479, -0.21140561],
[-1.06394988, -1.1060044],
[1.48121236, 1.09692001],
[-0.61475495, 0.70735537],

[0.98756515, -0.48304935],
[-1.83146213, 1.65415861],
[1.2802609 , 1.26189357],
[0.87514966, -0.99724226],
[-0.76692264, 0.80887407],
[-0.95788321, 0.96364573],
[1.7225576 , -0.03969445],
[0.69755426, -1.59831973],
[1.24434403, -0.51755318],
[-1.1364064 , -1.06988441],
[0.03928841, 1.21077994],
[1.01319972, -0.51148343],
[2.45547629, 0.51063543],
[-1.39570836, 1.32187833],
[-0.12078375, -0.35382094],
[-1.9225896 , -0.91129204],
[-2.63075707, 2.26481919],
[-0.98585459, -1.98218755],
[-1.58104657, 1.43812973],
[1.00704707, -1.18221929],
[0.76836754, -1.30328782],
[0.29813816, 1.8725563],
[0.88545073, -1.05526374],
[-1.30458637, -1.45816139],
[0.62361507, -1.72361803],
[-0.19566767, 0.35926855],
[-0.68340464, -0.62629264],
[-1.92428912, 1.72888979],
[-2.50657928, 2.19002644],
[-1.37218303, -1.66355965],
[0.87021729, -1.37907065],
[-0.282155 , -0.68047711],
[-0.58010955, 0.66354565],
[1.35226984, -0.14319613],
[-0.38138683, 0.51398183],
[-1.04571549, -1.17872083],
[0.0622821 , 0.17216823],
[-0.05001953, 0.26027914],
[-0.74418742, 0.81728693],
[2.08853625, 0.73487286],
[0.77352234, -1.61011319],
[0.24419984, 0.03151197],
[-1.86041923, -0.51616852],
[-1.26540606, -0.822232],
[-1.00038005, -0.76313421],
[-1.39603886, 1.3205836],

[0.32106058, 1.64401744],
[-1.43255648, -1.97300501],
[0.91110001, -0.4991374],
[-0.8320814 , 0.8581515],
[-0.51298131, 0.61788347],
[-1.56711185, 1.43476985],
[1.35770366, -0.41004841],
[0.00594443, 2.30931853],
[-0.87968303, 0.92491347],
[-1.3484831 , 1.74030154],
[1.56944674, 0.7403995],
[0.58978161, 1.6990815],
[-1.210668 , -0.65640423],
[-1.61061969, 1.4720785],
[-1.58743743, -1.43172805],
[1.48602265, 0.73227609],
[-1.34534184, -1.49550741],
[1.40273904, -0.29856663],
[0.75959857, -1.65489608],
[-0.17426253, 0.35451254],
[0.69558674, -1.78801737],
[0.68744542, -1.65035684],
[0.48920488, 1.33694379],
[-1.33746485, -0.95401574],
[-0.25596796, -0.84313575],
[0.66761917, 0.77217976],
[1.31798827, -0.33181096],
[2.76365918, 0.82625557],
[1.55275345, 1.71971969],
[3.94734569, -0.01015735],
[-1.8603884 , 0.02734122],
[-1.33850656, -0.21635023],
[-1.07872926, 1.06744975],
[0.72914878, -1.61481769],
[0.76055908, -1.44111171],
[1.61714586, -0.46088908],
[-0.51414998, 0.60979204],
[-1.09058817, -0.48726115],
[1.9576685 , -0.62234906],
[1.33809333, -0.44007144],
[0.67381392, -1.76878627],
[1.24809861, -0.02543529],
[-2.44167307, 2.1461878],
[-0.27881945, -1.14845206],
[-1.00374213, 1.00919515],
[0.33394736, 1.38716394],

[-0.27974225, -1.28792988],
[-0.57423336, -0.29204276],
[0.05652216, 0.17067635],
[1.13436022, -0.83600338],
[0.13791937, -2.45295666],
[-1.03857822, 1.04807785],
[1.36349422, -0.47856848],
[2.15782493, -0.39417194],
[-1.66241329, -0.04829992],
[-1.93247097, 0.78028182],
[1.16103909, -0.85991595],
[-0.44765252, 1.09033718],
[-1.34495779, -1.96193484],
[-1.52029316, -1.92436376],
[1.25467233, 1.1270591],
[-0.55089085, 0.64302336],
[1.84019141, 1.27475195],
[-0.10919934, 0.2985477],
[1.09623304, -0.55389211],
[-1.46032569, -0.68164209],
[1.00066577, -1.2995442],
[-0.14497356, -2.71271777],
[0.73203577, 1.84461543],
[0.89588609, -1.12409593],
[-0.97114965, 0.17995005],
[0.55253873, -1.71649366],
[0.68038333, -1.67985442],
[0.16398255, 1.33199271],
[0.20513002, 0.7254497],
[-0.70938114, 0.77186366],
[-1.17017813, -0.97624397],
[-1.56223124, 1.41339396],
[1.2161805 , -0.58189492],
[-0.77218664, -2.37652611],
[-0.81673563, 0.84490416],
[-0.90509801, -1.82853852],
[0.98164413, 1.32572036],
[-1.53481253, -0.19415763],
[0.87703457, -1.41915498],
[-1.3614981 , -0.31326299],
[-0.26766153, 0.42716331],
[0.21338121, 0.03266157],
[-1.40456368, -0.7640355],
[-0.11229819, -1.94453741],
[-0.39968662, 0.75014533],
[-1.05552164, -0.50701486],

```
       [ 0.50564412,  0.65475726],
       [ 1.27951498,  1.63967242],
       [ 0.720777  ,  1.67219376],
       [ 0.21258438,  0.05564996],
       [ 1.10865289,  0.34256539],
       [ 0.88049383, -0.95329753],
       [ 0.07811243,  2.04351838]])
```

This cell splits the data and selects RBF centers.
- Splits the dataset into training and testing sets (70% train, 30% test).
- Randomly selects 10 RBF centers from the training data.
- Sets the RBF width parameter `sigma` to 1.0.
- Prints the indices and shape of the chosen centers.

```
# Split the data before using x_train
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3,
random_state=42)
n_centers = 10
rng = np.random.default_rng(0)

center_indices = rng.choice(
    x_train.shape[0],
    n_centers,
    replace=False
)

rbf_centers = x_train[center_indices]
sigma = 1.0  # Standard deviation for RBF

print("center_indices: ",center_indices)
print("rbf_centers shape: ",rbf_centers.shape)
```

```
center_indices:  [170 209 128 103  55   8   3  63  36  15]
rbf_centers shape:  (10, 2)
```

```
scaler = StandardScaler()
x = scaler.fit_transform(x)
```

```
def gaussian_rbf(x_in, centers, sigma):

    dist_sq = cdist(x_in, centers, metric='sqeuclidean')
    phi = np.exp(- dist_sq / (2 * (sigma ** 2)))

    return phi

def rbf_layer(x_in, centers, sigma):
    return gaussian_rbf(x_in, centers, sigma)
```

```
# sanity check
phi_preview = rbf_layer(x_train[:5], rbf_centers, sigma)
print("phi_preview shape (5 samples -> n_centers):", phi_preview.shape)
```

phi_preview shape (5 samples -> n_centers): (5, 10)

```
phi_train = rbf_layer(x_train, rbf_centers, sigma)
```

```
model = LinearRegression()
model.fit(phi_train, y_train)
```

This cell tests the RBF network and computes accuracy.
- Transforms the test data into RBF feature space.
- Uses the trained model to predict continuous outputs, then thresholds them to get class predictions.
- Calculates and prints the test accuracy.

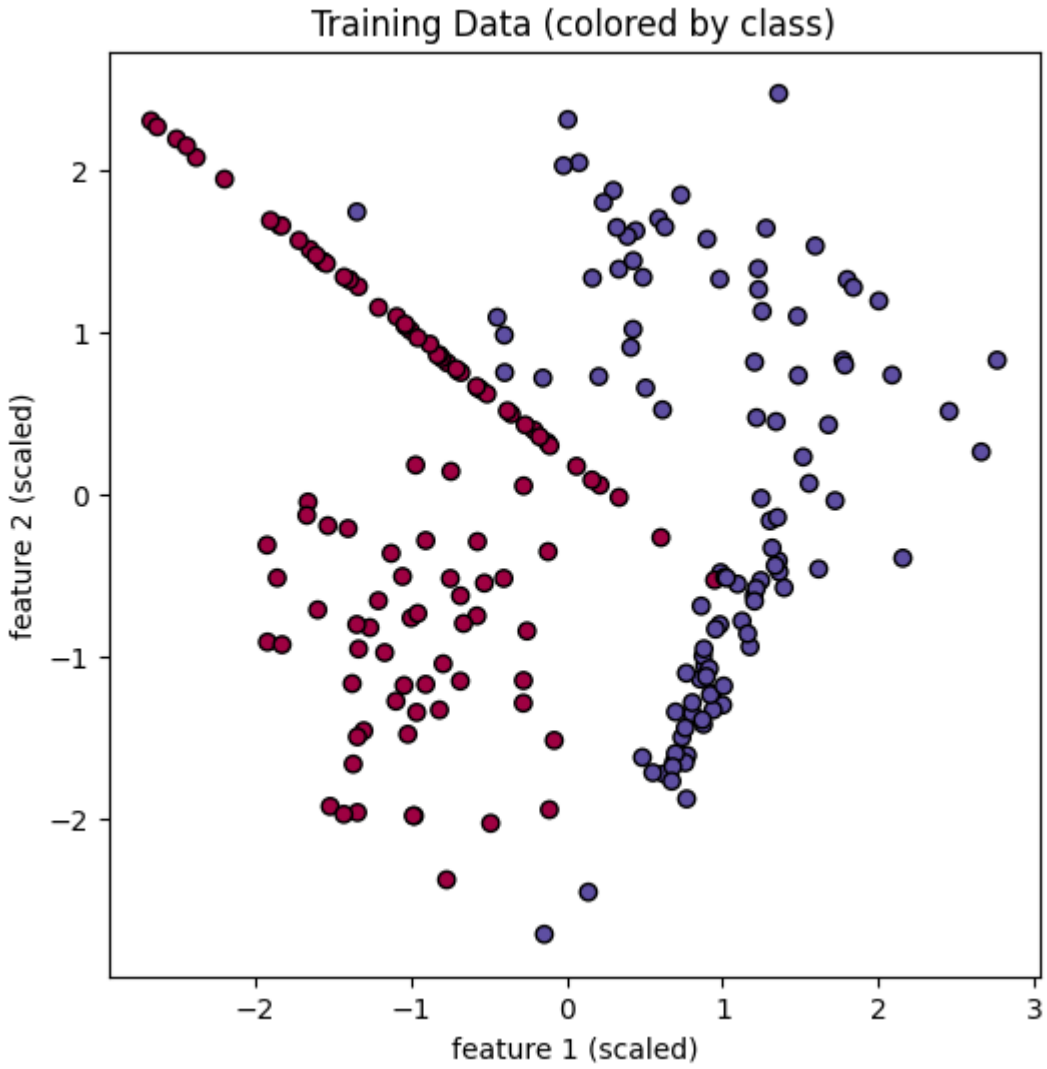
```
phi_test = rbf_layer(x_test, rbf_centers, sigma)
y_pred_cont = model.predict(phi_test)
y_pred_class = (y_pred_cont >= 0.5).astype(int)

acc = accuracy_score(y_test, y_pred_class)
print(f"Test accuracy: {acc * 100:.2f}%")
```

Test accuracy: 95.56%

This cell visualizes the training data.
- Plots the training samples in feature space, colored by their class labels.
- Helps you see the distribution of the two classes.

```
plt.figure(figsize=(6,6))
plt.scatter(
    x_train[:, 0],
    x_train[:, 1],
    c=y_train,
    cmap=plt.cm.Spectral,
    edgecolor="black"
)
plt.title("Training Data (colored by class)")
plt.xlabel("feature 1 (scaled)")
plt.ylabel("feature 2 (scaled)")
plt.show()
```



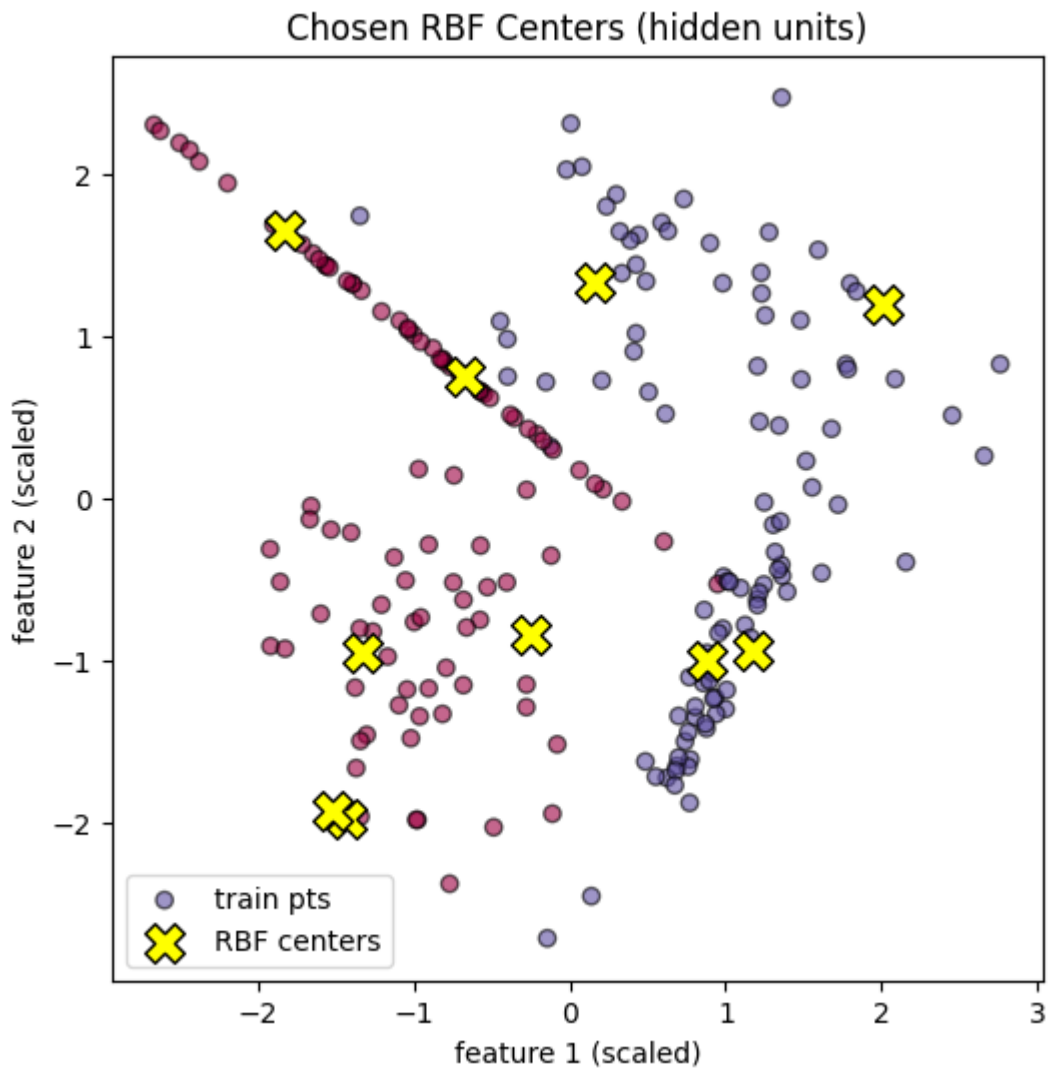
This cell visualizes the chosen RBF centers.
- Plots the training data and highlights the selected RBF centers as yellow X markers.
- Shows which points are used as the basis functions in the hidden layer.

```
plt.figure(figsize=(6,6))

# plot training data
plt.scatter(
    x_train[:, 0],
    x_train[:, 1],
    c=y_train,
    cmap=plt.cm.Spectral,
    edgecolor="black",
    alpha=0.6,
    label="train pts"
)
```

```python
# plot chosen RBF centers
plt.scatter(
    rbf_centers[:, 0],
    rbf_centers[:, 1],
    marker="X",
    s=200,
    c="yellow",
    edgecolor="black",
    label="RBF centers"
)

plt.title("Chosen RBF Centers (hidden units)")
plt.xlabel("feature 1 (scaled)")
plt.ylabel("feature 2 (scaled)")
plt.legend()
plt.show()
```



This cell plots the RBF network's decision boundary.
- Creates a grid of points and predicts their class using the trained RBF network.
- Plots the decision regions, all data points, and RBF centers.
- The plot title shows the test accuracy achieved by the model.

```
x_min, x_max = x[:, 0].min() - 1.0, x[:, 0].max() + 1.0
y_min, y_max = x[:, 1].min() - 1.0, x[:, 1].max() + 1.0

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

# flatten grid to list of points, like [[x1,y1],[x2,y2],...]
grid_points = np.c_[xx.ravel(), yy.ravel()]

# RBF features for every grid point
phi_grid = rbf_layer(grid_points, rbf_centers, sigma)

# Predict on grid
z_cont = model.predict(phi_grid)
z_class = (z_cont >= 0.5).astype(int)
z_class = z_class.reshape(xx.shape)  # reshape back to 2D so we can contour
plot

plt.figure(figsize=(6,6))

# background classification regions
plt.contourf(xx, yy, z_class, alpha=0.3, levels=[-1, 0, 1])

# plot all original samples
plt.scatter(
    x[:, 0],
    x[:, 1],
    c=y,
    cmap=plt.cm.Spectral,
    edgecolor="black"
)

# plot RBF centers again
plt.scatter(
    rbf_centers[:, 0],
    rbf_centers[:, 1],
    marker="X",
    s=200,
    c="yellow",
    edgecolor="black",
    label="RBF centers"
```

```
)

plt.title(f"RBF Network Decision Boundary\nTest Accuracy = {acc*100:.2f}%")
plt.xlabel("feature 1 (scaled)")
plt.ylabel("feature 2 (scaled)")
plt.legend()
plt.show()
```